

CHAPTER 1.2

STATISTICAL PATTERN RECOGNITION

KEINOSUKE FUKUNAGA

*School of Electrical Engineering, Purdue University
West Lafayette, IN 47907, USA*

In the introductory Section 1, the problems of statistical pattern recognition are defined, and a flow chart is presented to show how a classifier ought to be designed. In Section 2, the theoretically optimal (Bayes) classifier and its variations are introduced. The discussion is extended to show how the resulting classification (Bayes) error can be computed for some limited cases. Also, the upper and lower bounds of the Bayes error are shown. Section 3 discusses how a classifier is designed in practice. Linear, quadratic and piecewise classifiers are included. These are based on the expected vectors and covariance matrices of underlying probability distributions. In practice, these vectors and matrices are not given, and must be estimated from an available set of samples. Consequently, the designed classifier variates and the classification error becomes a random variable. Section 4 discusses how the number of available samples affects the classification performance, and also how to allocate the samples into design and test. In Section 5, nonparametric techniques are presented. They are needed in the estimation of the Bayes error and the structure analysis of data, where a mathematical formula such as Gaussianness cannot be applied. Both the Parzen and k nearest neighbor approaches are discussed. Feature extraction and clustering are discussed in other chapters.

Keywords: Statistical pattern recognition, classifier, probability of error, hypothesis tests, effect of sample size, nonparametric.

1. Introduction

The purpose of statistical pattern recognition is to determine to which category or class a given sample belongs. Through observation and measurement processes, we obtain a set of numbers which make up the measurement vector. The vector is a random vector and its density function depends on its class.

The design of a classifier consists of two parts. One is to collect data samples from various classes and to find the boundaries which separate the classes. This process is called *classifier design, training, or learning*. The other is to *test* the designed classifier by feeding the samples whose class identities are known.

Figure 1 shows a flow chart of how a classifier is designed [1]. After data is gathered, samples are normalized and registered. Normalization and registration are very important processes for a successful classifier design. However, different data require different normalization and registration, and it is difficult to discuss

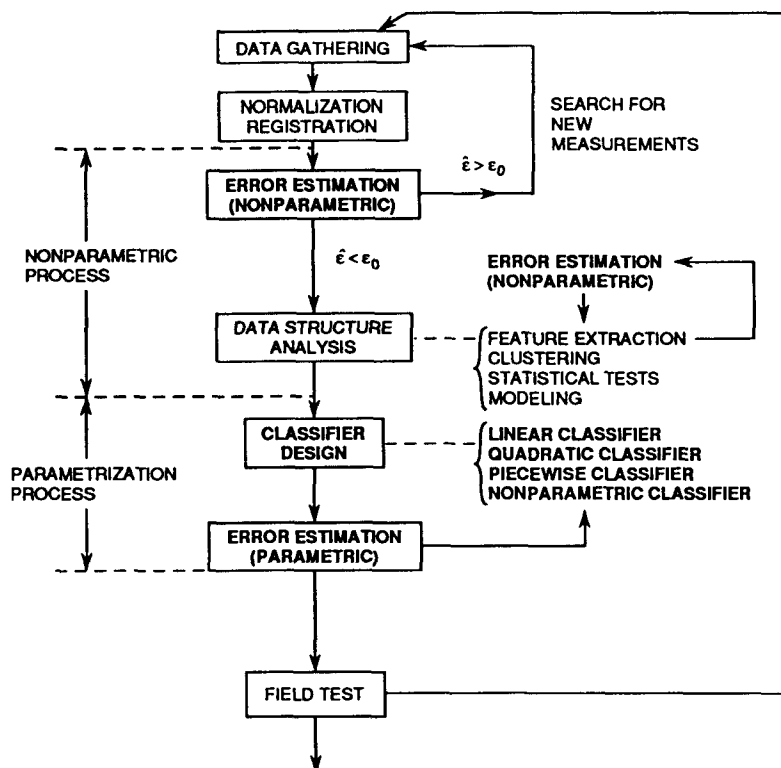


Fig. 1. Process of designing a classifier. (From [1], reprinted with permission.)

these subjects in a generalized way. Therefore, these subjects are not included in this chapter.

After normalization and registration, the class separability of the data is measured. This is done by estimating the *Bayes error*, the overlap among different class densities, in the measurement space. Since it is not appropriate at this stage to assume a mathematical form for the data structure, the estimation procedure must be *nonparametric*. If the Bayes error is larger than the final classifier error we wish to achieve (denoted by ϵ_0), it means the data does not carry enough classification information to meet the specification. Selecting features and designing a classifier in the later stages merely increase the classification error. Therefore, we must go back to data gathering and seek better measurements.

Only when the estimate of the Bayes error is less than ϵ_0 , may we proceed to the next stage of data structure analysis in which we study the characteristics of the data. All kinds of data analysis techniques are used here. They include feature extraction, clustering, statistical tests, modeling, and so on. Note that each time a

feature set is chosen, the Bayes error in the feature space is estimated and compared with the one in the measurement space. The difference between them indicates how much classification information is lost in the feature selection process.

Once the structure of the data is thoroughly understood, the data dictate which classifier must be adopted. Our choice is normally either a linear, quadratic, or piecewise classifier, and rarely a nonparametric classifier. Nonparametric techniques are required in off-line analyses to carry out many important operations such as the estimation of the Bayes error and data structure analysis. However, they are often too complex for any on-line operation.

After a classifier is designed, the classifier must be evaluated. The resulting error is compared with the Bayes error in the feature space. The difference between these two errors indicates how much the error is increased by adopting the classifier. If the difference is unacceptably high, we must re-evaluate the design of the classifier.

At last, the classifier is tested in the field. If the classifier does not perform as was expected, the database used for designing the classifier is different from the test data in the field. Therefore, we must expand the database and design a new classifier.

In this chapter, only the boldfaced portions of Fig. 1 will be discussed briefly. More details are found in [1]. Clustering and feature extraction are discussed in other chapters. Also, unless otherwise stated, only the two-class problem is discussed in this chapter, although the results for a more general multi-class problem are listed whenever available.

The notations, which are frequently used in this chapter, are summarized as follows:

Notation	
n	Dimensionality
L	Number of classes
N	Number of total samples
N_i	Number of class i samples
ω_i	Class i
P_i	<i>A priori</i> probability of ω_i
$X = [x_1 x_2 \dots x_n]^T$	Vector
$\mathbf{X} = [\mathbf{x}_1 \mathbf{x}_2 \dots \mathbf{x}_n]^T$	Random vector
$p_i(X) = p(x_1, x_2, \dots, x_n)$	Conditional density function of ω_i
$p(X) = \sum_{i=1}^L P_i p_i(X)$	Density function
$q_i(X) = P_i p_i(X)/p(X)$	<i>A posteriori</i> probability of ω_i given X
$M_i = E\{\mathbf{X} \omega_i\}$	Expected vector of ω_i
$M = E\{\mathbf{X}\} = \sum_{i=1}^L P_i M_i$	Expected vector
$\Sigma_i = E\{(\mathbf{X} - M_i)(\mathbf{X} - M_i)^T \omega_i\}$	Covariance matrix of ω_i
$\Sigma = E\{(\mathbf{X} - M)(\mathbf{X} - M)^T\}$	
$= \sum_{i=1}^L \{P_i \Sigma_i + P_i (M_i - M)(M_i - M)^T\}$	Covariance matrix

2. The Bayes Classification

In this section, the classification algorithms and the resulting errors are presented, based on the assumption that $p_i(X)$ and P_i are known. The classification algorithms are also known as *hypothesis tests*.

2.1. Likelihood Ratio Classifier

The probability of the classification error can be minimized by classifying X into either ω_1 or ω_2 , depending on whether $q_1(X) > q_2(X)$ or $q_1(X) < q_2(X)$ is satisfied. That is,

$$q_1(X) \underset{\omega_2}{\overset{\omega_1}{\geq}} q_2(X) \quad (\text{Bayes classifier}). \quad (2.1)$$

The resulting risk at X is

$$r^*(X) = \min[q_1(X), q_2(X)] \quad (\text{Bayes risk}). \quad (2.2)$$

The overall error is obtained by taking the expectation of (2.2) over $\mathbf{X}$:

$$\varepsilon^* = E\{r^*(\mathbf{X})\} = P_1 \int_{\Gamma_2} p_1(X) dX + P_2 \int_{\Gamma_1} p_2(X) dX \quad (\text{Bayes error}) \quad (2.3)$$

where $\varepsilon_1 = \int_{\Gamma_2} p_1(X) dX$ and $\varepsilon_2 = \int_{\Gamma_1} p_2(X) dX$ are called the ω_1 and ω_2 errors, respectively. Γ_i is the region where X is classified to ω_i .

For the multiclass problem,

$$q_k(X) = \max_i q_i(X) \rightarrow X \in \omega_k \quad (2.4)$$

$$\varepsilon^* = E \left\{ 1 - \max_i q_i(\mathbf{X}) \right\}. \quad (2.5)$$

A more convenient form of the Bayes classifier is obtained by applying the Bayes theorem, $q_i(X) = P_i p_i(X)/p(X)$, and taking negative logarithm:

$$h(X) = -\ln[p_1(X)/p_2(X)] \underset{\omega_2}{\overset{\omega_1}{\leq}} \ln[P_1/P_2] \quad (2.6)$$

The $h(X)$ combined with a threshold is called the *likelihood ratio classifier*.

When $\mathbf{X}$ is distributed Gaussianly with M_i and Σ_i for ω_i ,

$$-\ln p_i(X) = \frac{1}{2} (X - M_i)^T \Sigma_i^{-1} (X - M_i) + \frac{1}{2} \ln |\Sigma_i| + \left(\frac{n}{2} \right) \ln 2\pi. \quad (2.7)$$

The threshold of the classifier could be changed according to various requirements as follows:

Bayes classifier for minimum cost. Let c_{ji} be the cost of classifying a ω_i sample into ω_j . The expected cost of classifying X into ω_i is

$$c_i(X) = \sum_{j=1}^L c_{ji} q_j(X). \quad (2.8)$$

The classification rule and the resulting cost are

$$c_k(X) = \min_i c_i(X) \rightarrow X \in \omega_k \quad (2.9)$$

$$c^* = E \left\{ \min_i c_i(\mathbf{X}) \right\} . \quad (2.10)$$

For the two-class problem,

$$h(X) = -\ln[p_1(X)/p_2(X)] \underset{\omega_2}{\overset{\omega_1}{\gtrless}} \ln[(c_{12} - c_{11})P_1/(c_{21} - c_{22})P_2] . \quad (2.11)$$

This is a likelihood ratio classifier with a new threshold.

Neyman-Pearson test. Let ε_1 and ε_2 be the error probabilities from ω_1 and ω_2 , as shown in (2.3). The likelihood ratio classifier minimizes ε_1 subject to ε_2 being equal to a given constant, say, ε_0 . The threshold value must be selected to satisfy $\varepsilon_2 = \varepsilon_0$ and is normally determined empirically. A plot of ε_1 vs. ε_2 for the likelihood ratio classifier with varying threshold is called the *operating characteristics* and is used frequently as a visual aid to see how two errors are traded by changing the threshold. In the Neyman-Pearson test, $\varepsilon_2 = \varepsilon_0$ is the operating point and the corresponding threshold value is chosen. When ω_2 represents a target to be identified against the other class (ω_1), ε_1 , ε_2 , and $1 - \varepsilon_2$ are called the *false alarm*, the *leakage*, and the *detection probability*, respectively.

Minimax test. We can make the expected cost invariant even when P_i varies unexpectedly after the classifier has been implemented. This is done by selecting the threshold of the likelihood ratio classifier to satisfy

$$(c_{11} - c_{22}) + (c_{12} - c_{11})\varepsilon_1 - (c_{21} - c_{22})\varepsilon_2 = 0 . \quad (2.12)$$

Particularly, when $c_{11} = c_{22}$ and $c_{12} - c_{11} = c_{21} - c_{22}$, the threshold is chosen to satisfy $\varepsilon_1 = \varepsilon_2$. This classifier eliminates the possibility of having an unexpected large error due to the unexpected variation of P_i .

In all of these three cases, the likelihood ratio classifier is commonly used, and only the threshold varies. This may be interpreted as replacing the true P_i 's of (2.6) by artificial P_i 's. Therefore, theoretically, all these cases may be treated as the Bayes classifier, assigning the different meaning to P_i for each application.

Some other subjects related to hypothesis tests are as follows.

Independent measurement set. When $\mathbf{X}$ consists of statistically independent measurement sets as $\mathbf{X}^T = [\mathbf{X}_1^T \mathbf{X}_2^T \cdots \mathbf{X}_M^T]$, the Bayes classifier becomes

$$-\ln[p_1(X)/p_2(X)] = \sum_{i=1}^M -\ln[p_1(X_i)/p_2(X_i)] \underset{\omega_2}{\overset{\omega_1}{\gtrless}} \ln[P_1/P_2] \quad (2.13)$$

This suggests how to combine, for classification, seemingly unrelated information such as radar and infrared signatures.

One-class classifier. When one clearly defined class is classified against all other (sometimes not well-defined) possibilities, the boundary may be determined from the knowledge of one class only. A typical example is a hyperspherical boundary around a Gaussian distribution with $M = 0$ and $\Sigma = I$. This technique could work when the dimensionality of the data, n , is very low (such as 1 or 2). However, as n increases, the error of this technique increases significantly. The mapping from the original n -dimensional space to a one-dimensional distance space destroys valuable classification information which existed in the original space. For an example with $n = 64$, the error increases from 0.1% to 8.4%.

Reject. When X falls in the region where the Bayes risk $r^*(X)$ is high, we may decide not to classify the sample. This concept is called *reject*. The *reject region* $\Gamma_r(t)$ and the resulting *probability of rejection* $R(t)$ are specified by the threshold t as

$$\Gamma_r(t) = \{X : r^*(X) > t\} \quad (2.14)$$

$$R(t) = \Pr\{r^*(\mathbf{X}) > t\} = 1 - \Pr\{r^*(\mathbf{X}) \leq t\}. \quad (2.15)$$

Note that $\Pr\{r^*(\mathbf{X}) \leq t\}$ is the distribution function of a random variable $r^*(\mathbf{X})$. The probability of error with reject is the integration of $r^*(X)p(X)$ in $\bar{\Gamma}_r$, outside Γ_r , and thus depends on t . The error may be evaluated directly from the reject probability as

$$\varepsilon(t) = - \int_0^t \xi dR(\xi). \quad (2.16)$$

The error decreases as the reject probability increases, and vice versa. A plot of $\varepsilon(t)$ vs. $R(t)$ is called the *error-reject curve*, and is used as a visual aid to see how $\varepsilon(t)$ and $R(t)$ are traded by changing the threshold t . With the largest possible $t = 1 - 1/L$ for the L -class problem, $R(1 - 1/L) = 0$ and $\varepsilon(1 - 1/L)$ is equal to the Bayes error, ε^* .

Model validation. The distribution function of the random variable $r^*(\mathbf{X})$ is a simple and good parameter to characterize the classification environment, determining both $\varepsilon(t)$ and $R(t)$. Thus, when samples are drawn and a mathematical model is assumed, two distribution functions of $r^*(\mathbf{X})$ may be obtained: one empirically from the samples and the other theoretically from the model. The comparison of these two distribution functions could be used to test the validity of the mathematical model.

2.2. The Bayes Error

The Bayes error of (2.3) is generally hard to compute, except for the following two cases.

- (1) *Gaussian $\mathbf{X}$ with $\Sigma_1 = \Sigma_2 = \Sigma$.* For this case, the Bayes classifier becomes a linear function of X as

$$h(X) = (M_2 - M_1)^T \Sigma^{-1} X + \frac{1}{2} (M_1^T \Sigma^{-1} M_1 - M_2^T \Sigma^{-1} M_2) \leq t. \quad (2.17)$$

Since $\mathbf{X}$ is Gaussianly distributed, $h(\mathbf{X})$ is also a Gaussian random variable. Therefore,

$$\varepsilon_1 = \int_{(t-m_1)/\sigma_1}^{+\infty} \frac{1}{\sqrt{2\pi}} e^{-\xi^2/2} d\xi \text{ and } \varepsilon_2 = \int_{-\infty}^{(t-m_2)/\sigma_2} \frac{1}{\sqrt{2\pi}} e^{-\xi^2/2} d\xi \quad (2.18)$$

where

$$m_1 = E\{h(\mathbf{X})|\omega_1\} = -\frac{1}{2} (M_2 - M_1)^T \Sigma^{-1} (M_2 - M_1) \quad (2.19)$$

$$m_2 = E\{h(\mathbf{X})|\omega_2\} = +\frac{1}{2} (M_2 - M_1)^T \Sigma^{-1} (M_2 - M_1) \quad (2.20)$$

$$\sigma_i^2 = \text{Var}\{h(\mathbf{X})|\omega_i\} = (M_2 - M_1)^T \Sigma^{-1} (M_2 - M_1) \quad (i = 1, 2). \quad (2.21)$$

(2) *Gaussian $\mathbf{X}$ with $\Sigma_1 \neq \Sigma_2$.* The Bayes classifier for this case is

$$h(X) = \frac{1}{2} (X - M_1)^T \Sigma_1^{-1} (X - M_1) - \frac{1}{2} (X - M_2)^T \Sigma_2^{-1} (X - M_2) + \frac{1}{2} \ln(|\Sigma_1|/|\Sigma_2|) \leq t. \quad (2.22)$$

The distribution of $h(\mathbf{X})$ is no longer Gaussian, but the errors for a general $h(X)$ can be expressed as follows:

$$\varepsilon_1 = \int_{h(X) > t} p_1(X) dX = \frac{1}{2} + \frac{1}{2\pi} \iint \frac{e^{j\omega[h(X)-t]}}{j\omega} p_1(X) d\omega dX \quad (2.23)$$

$$\varepsilon_2 = \int_{h(X) < t} p_2(X) dX = \frac{1}{2} - \frac{1}{2\pi} \iint \frac{e^{j\omega[h(X)-t]}}{j\omega} p_2(X) d\omega dX \quad (2.24)$$

where the unspecified integral regions are the entire domain for X and $[-\infty, +\infty]$ for ω . Particularly, when $h(X)$ is the quadratic function of (2.22) and $p_i(X)$'s are Gaussian, we can integrate (2.23) and (2.24) explicitly with respect to X to obtain

$$\varepsilon_1 = \frac{1}{2} + \frac{1}{\pi} \int_0^\infty \frac{\prod_{i=1}^n |\phi_{1i}(\omega)|}{\omega} \sin \left\{ \left[\sum_{i=1}^n \angle \phi_{1i}(\omega) \right] - \omega t \right\} d\omega \quad (2.25)$$

$$\varepsilon_2 = \frac{1}{2} - \frac{1}{\pi} \int_0^\infty \frac{\prod_{i=1}^n |\phi_{2i}(\omega)|}{\omega} \sin \left\{ \left[\sum_{i=1}^n \angle \phi_{2i}(\omega) \right] - \omega t \right\} d\omega \quad (2.26)$$

where

$$|\phi_{ji}(\omega)| = (1 + \omega^2 a_{ji})^{-1/4} \exp \left[\frac{1}{2} (-b_{ji}^2 \omega^2) / (1 + \omega^2 a_{ji}^2) \right] \quad (2.27)$$

$$\angle \phi_{ji}(\omega) = \frac{1}{2} \tan^{-1}(a_{ji}\omega) - \frac{1}{2} \omega [c_{ji} + (a_{ji} b_{ji}^2 \omega^2) / (1 + \omega^2 a_{ji}^2)] \quad (2.28)$$

$$a_{1i} = 1 - \frac{1}{\lambda_i}, \quad b_{1i} = \frac{d_{2i} - d_{1i}}{\lambda_i}, \quad c_{1i} = \frac{b_{1i}^2}{1 - a_{1i}} + \ln \lambda_i \quad (2.29)$$

$$a_{2i} = \lambda_i - 1, \quad b_{2i} = \sqrt{\lambda_i} (d_{2i} - d_{1i}), \quad c_{2i} = \frac{-b_{2i}^2}{1 + a_{2i}} + \ln \lambda_i. \quad (2.30)$$

The λ 's are the diagonal components of Λ which are obtained by simultaneously diagonalizing Σ_1 and Σ_2 as

$$A^T \Sigma_1 A = I \quad \text{and} \quad A^T \Sigma_2 A = \Lambda \quad (2.31)$$

and the $(d_{2i} - d_{1i})$ is the i th component of the vector $A^T(M_2 - M_1)$. Equations (2.25) and (2.26) must be integrated numerically, but they are one-dimensional integrations.

Upper and lower bounds. The computation of the Bayes error is very complex unless $h(X)$ is linear. Furthermore, since a numerical integration is involved, the Bayes error cannot be expressed explicitly. An alternative is to use the *upper* and *lower bounds* of the Bayes error as a measure of class separability. Some of popular bounds are listed as follows:

$$E\{q_1(\mathbf{X})q_2(\mathbf{X})\} : \quad 2 \text{ nearest-neighbor error} \quad (2.32)$$

$$E\{\min[q_1(\mathbf{X}), q_2(\mathbf{X})]\} : \quad \text{Bayes error} \quad (2.33)$$

$$2E\{q_1(\mathbf{X})q_2(\mathbf{X})\} : \quad \text{Nearest-neighbor error} \quad (2.34)$$

$$-\frac{1}{2\ln 2} E\{q_1(\mathbf{X}) \ln q_1(\mathbf{X}) + q_2(\mathbf{X}) \ln q_2(\mathbf{X})\} : \quad \text{Equivocation} \quad (2.35)$$

$$E\{\sqrt{q_1(\mathbf{X})q_2(\mathbf{X})}\} : \quad \text{Bhattacharyya bound} \quad (2.36)$$

The inequalities $(2.32) \leq (2.33) \leq (2.34) \leq (2.35) \leq (2.36)$ hold regardless of the distributions.

One of the popular bounds is the *Bhattacharyya bound*, which has an explicit expression for *Gaussian distributions*:

$$E\{\sqrt{q_1(\mathbf{X})q_2(\mathbf{X})}\} = \sqrt{P_1 P_2} e^{-\mu} \quad (2.37)$$

$$\mu = \frac{1}{8} (M_1 - M_2)^T \left[\frac{\Sigma_1 + \Sigma_2}{2} \right]^{-1} (M_1 - M_2) + \frac{1}{2} \ln \frac{|\Sigma_1 + \Sigma_2|/2}{\sqrt{|\Sigma_1|}\sqrt{|\Sigma_2|}}. \quad (2.38)$$

The first term of (2.38) indicates the class separability due to the mean difference, and the second term gives that due to the covariance difference.

When the distributions are non-Gaussian, (2.37) with (2.38) is no longer guaranteed to bound the Bayes error. Still, in order to use (2.38) as an effective class separability measure, one may transform each variable to a Gaussian-like one. For example, *power transforms*, $y_i = x_i^\nu$ ($i = 1, \dots, n$), may be used for causal distributions. Such a variable transformation is useful not only for measuring the class separability, but also for designing a better classifier. Designing the Bayes classifier for Gaussian distributions, even with additional variable transformations, is often easier than designing the Bayes classifier for non-Gaussian distributions.

Scatter measures. The bounds discussed above are valid only for two class problems. There exist no well accepted extensions of the above bounds to multiclass. *Scatter measures*, introduced here as an alternative, are intuitively derived, simple, but not

directly related to the Bayes error. They are for multiclass. Let us define for L -class:

$$S_b = \sum_{i=1}^L P_i (M_i - M)(M_i - M)^T \quad \text{Between-class scatter matrix} \quad (2.39)$$

$$S_w = \sum_{i=1}^L P_i \Sigma_i \quad \text{Within-class scatter matrix} \quad (2.40)$$

$$S_m = E\{(\mathbf{X} - M)(\mathbf{X} - M)^T\} = S_b + S_w \quad \text{Mixture scatter matrix} \quad (2.41)$$

The class separability can be measured by the combinations of these matrices:

$$\text{tr} S_w^{-1} S_b, \quad \text{tr} S_m^{-1} S_w, \quad \ln |S_m^{-1} S_w| \quad \text{etc.} \quad (2.42)$$

where trace and log-determinant are for converting a matrix to a number. The first and second ones are often used for feature extraction and clustering respectively. All combinations, $S_w^{-1} S_b$, $S_m^{-1} S_w$, $S_m^{-1} S_b$ etc., share the same eigenvectors, and their eigenvalues are well related. The first one of (2.42) measures class separability based on the scatter of class means, normalized by S_w , and the others are the variations. Therefore, the scatter measures can be applied only for the cases where classes are separated mainly by mean-difference. There are no measures available for multiclass mainly separated by covariance-difference.

3. Classifier Design

Once the structure of data is studied thoroughly, it is easy to select a proper classifier for the data. This section presents how several typical classifiers can be designed.

3.1. Linear Classifiers

The Bayes classifier becomes linear for the following two cases.

- (1) *Gaussian $\mathbf{X}$ with $\Sigma_1 = \Sigma_2 = \Sigma$* : For this case, the Bayes classifier is expressed by (2.17), which is linear. In particular, when $\Sigma = I$,

$$h(X) = (M_2 - M_1)^T X + \frac{1}{2} (M_1^T M_1 - M_2^T M_2) \leq t. \quad (3.1)$$

This classifier is also known as the *distance classifier* in which $(X - M_1)^T (X - M_1) \stackrel{\omega_1}{\leq} (X - M_2)^T (X - M_2)$, or the *correlation classifier* in which $M_1^T X \stackrel{\omega_1}{\geq} M_2^T X$ with an energy constant condition $M_1^T M_1 = M_2^T M_2$. In both cases, $P_1 = P_2$ and thus $t = 0$ is assumed. When $\Sigma \neq I$, the distance or correlation classifier may still be applied, but only after $\mathbf{X}$ is linearly transformed to $\mathbf{Y} = A^T \mathbf{X}$ in order to make $\Sigma_Y = A^T \Sigma A = I$.

- (2) *Binary independent $\mathbf{x}_i$'s*: For independent $\mathbf{x}_i$'s taking either +1 or -1,

$$p_i(X) = \prod_{j=1}^n w_{ij}^{(1+x_j)/2} (1 - w_{ij})^{(1-x_j)/2} [\delta(x_j - 1) + \delta(x_j + 1)] \quad (3.2)$$

where $w_{ij} = \Pr\{x_j = +1|\omega_i\}$. Substituting (3.2) into (2.6),

$$h(X) = -\frac{1}{2} \left\{ \sum_{j=1}^n \left[\ln \frac{w_{1j}(1-w_{2j})}{w_{2j}(1-w_{1j})} \right] x_j + \sum_{j=1}^n \left[\ln \frac{w_{1j}(1-w_{1j})}{w_{2j}(1-w_{2j})} \right] \right\}. \quad (3.3)$$

For Gaussian $\mathbf{X}$ with $\Sigma_1 \neq \Sigma_2$ and more general non-Gaussian $\mathbf{X}$, a linear classifier is not the best one. However, because of its simplicity and robustness, a linear classifier is frequently adopted. The design procedure is as follows.

$$h(X) = V^T X \underset{\omega_2}{\overset{\omega_1}{\leq}} t. \quad (3.4)$$

Equation (3.4) indicates that $\mathbf{X}$ is linearly mapped down to a variable $\mathbf{h}$ and the distributions of $\mathbf{h}$ for ω_1 and ω_2 are separated by a threshold t . Thus, the optimum V is found by minimizing the probability of error in the h -space. Because of complexity in the error computation, simpler criteria such as $f(m_1, m_2, \sigma_1^2, \sigma_2^2)$ are often used, where $m_i = E\{h(\mathbf{X})|\omega_i\} = V^T M_i$ and $\sigma_i^2 = \text{Var}\{h(\mathbf{X})|\omega_i\} = V^T \Sigma_i V$. The typical examples are

$$f = \frac{(m_1 - m_2)^2}{\sigma_1^2 + \sigma_2^2} \quad \text{Fisher's criterion} \quad (3.5)$$

$$f = \frac{P_1(m_1 - m_0)^2 + P_2(m_2 - m_0)^2}{P_1\sigma_1^2 + P_2\sigma_2^2} \quad \left(\frac{\text{between class scatter}}{\text{within class scatter}} \right) \quad (3.6)$$

where $m_0 = P_1 m_1 + P_2 m_2$ is the mixture mean. These criteria measure the class separability of the distributions of $\mathbf{h}$. The solution of $\partial f / \partial V = 0$ is

$$V \propto [s\Sigma_1 + (1-s)\Sigma_2]^{-1}(M_2 - M_1) \quad (3.7)$$

where

$$s = \frac{\partial f / \partial \sigma_1^2}{\partial f / \partial \sigma_1^2 + \partial f / \partial \sigma_2^2}. \quad (3.8)$$

That is, the optimum V always takes the form of (3.7) regardless of the functional form of f . The effect of f is observed only in the averaging coefficient of covariance matrices, s . For example, $s = 0.5$ for (3.5) and $s = P_1$ for (3.6).

V can be found even without specifying f . Since the form of V is known as in (3.7), we change s from 0 to 1 with a certain increment, say, 0.1, compute the empirical distribution functions of $\mathbf{h} = V^T \mathbf{X}$ for ω_1 and ω_2 from the given data set, select the value of the threshold, and count the number of misclassified samples. The optimum s is the one which gives the smallest error in this operation.

The Bhattacharyya bound of (2.38) gives a simple test to decide whether or not a linear classifier is appropriate. When the first term of (2.38) is dominant, the classifiability comes mainly from the mean difference. Therefore, a linear classifier is

a proper choice. However, if the second term is significant, the covariance difference plays an important role, and a quadratic classifier is called for.

3.2. Quadratic Classifiers

For Gaussian $\mathbf{X}$, the Bayes classifier becomes quadratic, as shown in (2.7) or (2.22). In practice, the quadratic classifier of (2.22) is widely adopted in many applications, even without checking the Gaussianness of $\mathbf{X}$, and with much success. Probably, this is the classifier everyone may try first, even before conducting data structure analysis.

However, it is not known how to design the optimum quadratic classifier for non-Gaussian distributions, as the linear classifier was designed. The optimization of $f(m_1, m_2, \sigma_1^2, \sigma_2^2)$ for $\mathbf{h} = \mathbf{X}^T \mathbf{Q} \mathbf{X} + V^T \mathbf{X}$ with respect to a matrix $\mathbf{Q}$ and a vector V is too complex. If quadratic terms $\mathbf{x}_j \mathbf{x}_k$'s are treated as new variables $\mathbf{y}_i$, $\mathbf{h} = \Sigma \Sigma q_{jk} \mathbf{x}_j \mathbf{x}_k + \Sigma \nu_i \mathbf{x}_i$ becomes a linear equation as $\mathbf{h} = \Sigma a_i \mathbf{y}_i + \Sigma \nu_i \mathbf{x}_i$. However, for high-dimensional cases, the number of $\mathbf{y}_i$'s becomes prohibitively large.

Two-dimensional display. One of the procedures used to improve the performance of the quadratic classifier is to plot $\mathbf{X}$ in a two-dimensional display where $d_i^2(X) = (X - M_i)^T \Sigma_i^{-1} (X - M_i)$ for $i = 1, 2$ are used as the x and y axes. If $\mathbf{X}$ is Gaussian, the Bayes classifier becomes a 45° line with a proper y -crossing point. When the distribution is not perfectly Gaussian, we can observe in the display that the 45° line is not the best boundary to minimize the number of misclassified samples. Then, visually we can find a better line to classify samples by changing the slope and the y -crossing point of the line. It corresponds to adjusting α and β of the following quadratic classifier:

$$d_2^2(X) \stackrel{\omega_1}{\leq} \alpha d_1^2(X) + \beta. \quad (3.9)$$

Once samples are plotted and examined, the boundary in the display need not be restricted to a line. Any curve could be drawn. This flexibility is the advantage of seeing the data on the display.

Fourier transform. When a *stationary* random process is time-sampled, the coefficients of the *discrete Fourier transform* are uncorrelated, and its covariance matrix becomes diagonal. Thus, the quadratic classifier of the Fourier coefficients $\mathbf{y}_i$ is reduced to $\mathbf{h} = \Sigma_{i=1}^n (q_i |\mathbf{y}_i|^2 + v_i \mathbf{y}_i) + v_0$. This is the Bayes classifier, if $\mathbf{y}_i$'s are Gaussian.

Approximation of covariance matrices. If we can assume a structure for a covariance matrix, we can simplify the design of a quadratic classifier. In addition, the classifier becomes less sensitive to the parameter variation due to the estimation process by using a finite number of design samples. This will be discussed in the next section.

One of the possible structures is the *toeplitz form* for the correlation matrix, allowing each variable to have its distinct variance. In this case, parameters must be selected to assure that the toeplitz matrix be positive-definite. In particular, when

the correlation coefficient between the i th and j th variables, γ_{ij} , is approximated by $\rho^{|i-j|}$ (ρ : a constant), the entire correlation matrix is characterized by one parameter, ρ , and its determinant and inverse matrix may be expressed explicitly.

3.3. Piecewise Classifiers

For the *multiclass problem*, the boundary must have a piecewise structure as follows.

Piecewise quadratic classifiers. If $\mathbf{X}$ is Gaussian, the classifier becomes, from (2.7), piecewise quadratic:

$$\min_i \left[\frac{1}{2} (\mathbf{X} - \mathbf{M}_i)^T \Sigma_i^{-1} (\mathbf{X} - \mathbf{M}_i) + \frac{1}{2} \ln |\Sigma_i| - \ln P_i \right]. \quad (3.10)$$

The first term of (3.10) is widely used even for non-Gaussian distributions. However, it must be noted that the normalized distance of $\mathbf{X}$ from each class mean, $\mathbf{M}_i$, must be adjusted by two constant terms, $\frac{1}{2} \ln |\Sigma_i|$ and $\ln P_i$.

Piecewise linear classifiers. When Σ_i 's are similar, the quadratic term $\mathbf{X}^T \Sigma_i^{-1} \mathbf{X}$ is eliminated from (3.10) to get a piecewise linear classifier:

$$\min_i \left[-\mathbf{M}_i^T \Sigma_i^{-1} \mathbf{X} + \frac{1}{2} \mathbf{M}_i^T \Sigma_i^{-1} \mathbf{M}_i + \frac{1}{2} \ln |\Sigma_i| - \ln P_i \right]. \quad (3.11)$$

Or, replacing Σ_i by the averaged covariance, $\Sigma = (\Sigma_1 + \cdots + \Sigma_L)/L$,

$$\min_i \left[-\mathbf{M}_i^T \Sigma^{-1} \mathbf{X} + \frac{1}{2} \mathbf{M}_i^T \Sigma^{-1} \mathbf{M}_i - \ln P_i \right]. \quad (3.12)$$

Another possibility is to design the optimal linear classifier for each pair of classes. In this case, $L(L-1)/2$ classifiers must be designed, instead of L in (3.11) or (3.12).

Clustering. In some applications, each class distribution would be handled better by dividing it into several clusters. For example, take the signatures of a target viewed from the front and side. Since they are so different, it may be more appropriate to separate them into several clusters rather than to treat all of them as one class.

Considering each cluster as a class, we can form a new multiclass problem with a significantly increased number of classes. However, the details of designing such a classifier depend very much on how clusters are defined and obtained and how many classes are generated. Therefore, although important, the subject is not discussed in this chapter.

k nearest neighbor (kNN). The k NN classifier forms a piecewise linear boundary, although it is very complex and data dependent. The simpler boundary could be obtained by merging samples into a smaller number of representatives and then applying the k NN classifier to these representatives.

3.4. Sequential Classifiers

When m consecutive observation vectors, $X_1, \dots, X_m$, are known as coming from the same class, we can use this additional information to reduce the classification error. That is, the number of variables is extended from n for one vector to $m \times n$ for m vectors. Thus, we can form a new random vector with $m \times n$ components and design a classifier in the $(m \times n)$ -dimensional space. However, when these vectors are *statistically independent*, a simpler formula could be adopted:

$$-\ln[p_1(X_1, \dots, X_m)/p_2(X_1, \dots, X_m)] = \sum_{i=1}^m \{-\ln[p_1(X_i)/p_2(X_i)]\} \stackrel{\omega_1}{\omega_2} \leq t. \quad (3.13)$$

That is, the likelihood ratio classifier is applied to the incoming sample X_i , and the output is accumulated. Rewriting the left-hand side of the inequality as

$$\mathbf{s} = \frac{1}{m} \sum_{i=1}^m h(\mathbf{X}_i) \quad (3.14)$$

the expected values and variances of $\mathbf{s}$ and $\mathbf{h} = h(\mathbf{X})$ are related by

$$E\{\mathbf{s}|\omega_i\} = E\{\mathbf{h}|\omega_i\} \quad \text{and} \quad \text{Var}\{\mathbf{s}|\omega_i\} = \frac{1}{m} \text{Var}\{\mathbf{h}|\omega_i\}. \quad (3.15)$$

Thus, we can reduce the variances of $\mathbf{s}$ by increasing m , while maintaining the expected values of $\mathbf{s}$. Furthermore, the density function of $\mathbf{s}$ becomes close to a Gaussian by the central limit theorem.

Two important properties of the sequential classifier emerge from the above discussion. One is that we can make the error as small as we like by increasing m . The other is that the error is determined by a small number of parameters, $E\{\mathbf{h}|\omega_i\}$, $\text{Var}\{\mathbf{h}|\omega_i\}$, and m , and is little affected by the higher-order moments of $\mathbf{h}$.

In practice, the true $p_i(X)$'s are never known, and $h(X) = -\ln[p_1(X)/p_2(X)]$ must be replaced by some function $\hat{h}(X)$. A desired property for $\hat{h}(X)$ is

$$E\{\hat{h}(\mathbf{X})|\omega_1\} \leq 0 \quad \text{and} \quad E\{\hat{h}(\mathbf{X})|\omega_2\} \geq 0. \quad (3.16)$$

regardless of the distribution of $\mathbf{X}$. As long as (3.16) is satisfied, the random variable $\hat{h}(\mathbf{X})$ carries classification information, however small, regardless of the distribution of $\mathbf{X}$. The classifiable information can be enhanced as much as we like by increasing m in the sequential operations. Two $\hat{h}(X)$'s are known to satisfy (3.16) for all distributions of $\mathbf{X}$, whose expected vectors and covariance matrices are M_1 and Σ_1 for ω_1 and M_2 and Σ_2 for ω_2 :

$$\hat{h}(X) = (M_2 - M_1)^T \Sigma^{-1} X + \frac{1}{2} (M_1^T \Sigma^{-1} M_1 - M_2^T \Sigma^{-1} M_2) \quad (3.17)$$

$$\begin{aligned} \hat{h}(X) = & \frac{1}{2} (X - M_1)^T \Sigma_1^{-1} (X - M_1) - \frac{1}{2} (X - M_2)^T \Sigma_2^{-1} (X - M_2) \\ & + \frac{1}{2} \ln(|\Sigma_1|/|\Sigma_2|). \end{aligned} \quad (3.18)$$

Any positive-definite matrix Σ in (3.17) satisfies (3.16). But, the averaged covariance matrix such as the one in (3.7) would be a better Σ to achieve the same performance with a smaller m . Equation (3.17) could be used, if the first term is dominant in the Bhattacharyya bound (2.38), but (3.18) is more appropriate otherwise. Note that these equations are the same as (2.17) and (2.22), respectively.

One of the most important aspects in classifier design is to make the classifier *robust*. That is, the performance of the classifier must be maintained, even if the distribution of test samples becomes somewhat different from the one used for design. The sequential technique can compensate the degradation of the performance of $h(X_i) \leq t$ by increasing m .

4. Estimation of Classification Errors

So far, we have discussed the design of parametric classifiers, assuming that M_i and Σ_i are given. In practice, these parameters are estimated by using a finite number of available samples, and the estimates are random variables. Consequently, the classifier designed with these estimates varies, and its performance is also random. Therefore, it is important to know how the sample size affects classifier design and performance.

4.1. Effect of Sample Size on Estimation

General formula. First let us consider the problem of estimating $f = f(y_1, \dots, y_q)$ by $\hat{f} = f(\hat{y}_1, \dots, \hat{y}_q)$, where f is a given function, the y_i 's are the true parameter values, and $\hat{y}_i$'s are their estimates. When the deviation of $\hat{y}_i$ from y_i is small, $\hat{f}$ may be expanded by a Taylor series as follows:

$$\hat{f} \cong f + \sum_{i=1}^q \frac{\partial f}{\partial y_i} \Delta y_i + \frac{1}{2} \sum_{i=1}^q \sum_{j=1}^q \frac{\partial^2 f}{\partial y_i \partial y_j} \Delta y_i \Delta y_j \quad (4.1)$$

where $\Delta y_i = \hat{y}_i - y_i$. If the estimates are *unbiased*,

$$E\{\hat{f}\} \cong f + \frac{1}{2} \sum_{i=1}^q \sum_{j=1}^q \frac{\partial^2 f}{\partial y_i \partial y_j} E\{\Delta y_i \Delta y_j\} \quad (4.2)$$

$$\text{Var}\{\hat{f}\} \cong \sum_{i=1}^q \sum_{j=1}^q \frac{\partial f}{\partial y_i} \frac{\partial f}{\partial y_j} E\{\Delta y_i \Delta y_j\}. \quad (4.3)$$

In most parametric cases, the y_i 's are the components of M_r and Σ_r ($r = 1, 2$), and $Y^T = [y_1 \cdots y_q]$ can be expressed as

$$Y^T = [m_1^{(1)}, \dots, m_n^{(1)}, m_1^{(2)}, \dots, m_n^{(2)}, c_{11}^{(1)}, \dots, c_{nn}^{(1)}, c_{11}^{(2)}, \dots, c_{nn}^{(2)}] \quad (4.4)$$

where $m_i^{(r)}$ and $c_{ij}^{(r)}$ ($i \leq j$) are the components of M_r and Σ_r respectively. Their unbiased estimates are obtained by the *sample mean* and *sample covariance matrix*

as

$$\hat{\mathbf{M}}_r = \frac{1}{N_r} \sum_{i=1}^{N_r} \mathbf{X}_i^{(r)} \quad \text{and} \quad \hat{\Sigma}_r = \frac{1}{N_r - 1} \sum_{i=1}^{N_r} (\mathbf{X}_i^{(r)} - \hat{\mathbf{M}}_r)(\mathbf{X}_i^{(r)} - \hat{\mathbf{M}}_r)^T \quad (4.5)$$

where $\mathbf{X}_i^{(r)}$ is the i th sample from ω_r . When $\mathbf{X}_i^{(r)}$'s are drawn from Gaussian distributions, $E\{\Delta \mathbf{y}_i \Delta \mathbf{y}_j\}$ for the y_i 's of (4.4) are known, and (4.2) and (4.3) become

$$E\{\Delta \mathbf{f}\} \cong \frac{1}{2N} \sum_{r=1}^2 \left[\sum_{i=1}^n \frac{\partial^2 f}{\partial m_i^{(r)2}} \lambda_i^{(r)} + \sum_{i=1}^n \frac{\partial^2 f}{\partial c_{ii}^{(r)2}} 2\lambda_i^{(r)2} + \sum_{i=1}^n \sum_{j=1}^{i-1} \frac{\partial^2 f}{\partial c_{ij}^{(r)2}} \lambda_i^{(r)} \lambda_j^{(r)} \right] \quad (4.6)$$

$$\begin{aligned} \text{Var}\{\hat{\mathbf{f}}\} \cong & \frac{1}{N} \sum_{r=1}^2 \left[\sum_{i=1}^n \left(\frac{\partial f}{\partial m_i^{(r)}} \right)^2 \lambda_i^{(r)} + \sum_{i=1}^n \left(\frac{\partial f}{\partial c_{ii}^{(r)}} \right)^2 2\lambda_i^{(r)2} \right. \\ & \left. + \sum_{i=1}^n \sum_{j=1}^{i-1} \left(\frac{\partial f}{\partial c_{ij}^{(r)}} \right)^2 \lambda_i^{(r)} \lambda_j^{(r)} \right] \end{aligned} \quad (4.7)$$

where both Σ_1 and Σ_2 are assumed to be diagonal with $\lambda_i^{(r)}$ ($r = 1, 2$) as their diagonal components. Without loss of generality, any two covariance matrices can be simultaneously diagonalized. Also, $N_1 = N_2 = N$ is assumed for simplicity. Note that both the bias of (4.6) and the variance of (4.7) are proportional to $1/N$. The other terms are determined by the underlying distributions. This is true even when $\mathbf{X}_i^{(r)}$'s are drawn from non-Gaussian distributions.

Estimation procedure of $\mathbf{f}$. Equation (4.6) can be rewritten as

$$E\{\hat{\mathbf{f}}\} \cong f + \frac{\nu}{N}. \quad (4.8)$$

This equation suggests the following procedure to estimate f :

- (1) Change N to $N_1, \dots, N_\ell$. For each N_i , compute $\hat{\mathbf{M}}_r$ and $\hat{\Sigma}_r$ and subsequently $\hat{\mathbf{f}}$. Repeat this τ times independently, and approximate $E\{\hat{\mathbf{f}}\}$ by the sample mean of the τ results.
- (2) Plot the empirical points of $E\{\hat{\mathbf{f}}\}$ vs. $1/N$. Then, find the line best fitted to these points. The slope of the line is ν , and the y -crossing point is the estimate of f .

Bhattacharyya distance. The Bhattacharyya distance of (2.38) is a function of M_r and Σ_r . Thus, the bias of (4.6) can be further reduced by computing the partial derivatives of this function. Treating the first and second terms separately,

$$E\{\Delta \mu_1\} \cong \frac{1}{4N} \left[n + \sum_{i=1}^n \sum_{j=1}^n \frac{m_j^2(1 + \lambda_i \lambda_j)}{(1 + \lambda_j)^2(1 + \lambda_i)} + \sum_{i=1}^n \frac{m_i^2(1 + \lambda_i^2)}{(1 + \lambda_i)^3} \right] \quad (4.9)$$

$$E\{\Delta \mu_2\} \cong \frac{1}{4N} \left[n(n+1) - \sum_{i=1}^n \sum_{j=1}^n \frac{1 + \lambda_i \lambda_j}{(1 + \lambda_i)(1 + \lambda_j)} - \sum_{i=1}^n \frac{1 + \lambda_i^2}{(1 + \lambda_i)^2} \right] \quad (4.10)$$

where $M_1 = 0$, $M_2 = [m_1 \dots m_n]^T$, $\lambda_i^{(1)} = 1$ ($\Lambda_1 = I$) and $\lambda_i^{(2)} = \lambda_i$ ($\Lambda_2 = \Lambda$) are used without losing generality. For example, when $m_i = 0$ and $\lambda_i = 1$, $E\{\Delta\mu_1\} \cong n/(4N)$ and $E\{\Delta\mu_2\} \cong n(n+1)/(8N)$. This is the case where the Bhattacharyya distance is measured between two sample sets generated from the same Gaussian distribution. Although $\mu_1 = \mu_2 = 0$ for $N = \infty$, a finite N creates the biases. Note that $E\{\Delta\mu_1\}$ is proportional to $1/k$ ($k = N/n$: ratio of sample size and dimensionality) while $E\{\Delta\mu_2\}$ depends on $(n+1)/k$. In order to maintain the same amount of bias ($E\{\Delta\mu\} = E\{\Delta\mu_1\} + E\{\Delta\mu_2\}$), a larger k must be chosen as n increases. For example, to meet $E\{\Delta\mu\} \leq 0.223$, k must be larger than 6.2 and 39.6 for $n = 8$ and 64 respectively. The variances of (4.7) also can be computed similarly.

4.2. Estimation of Classification Errors

The classification errors of (2.23) and (2.24) are the members of the family of functions presented in (4.1) and (4.4), when $h(X)$ and $p_r(X)$ are functions of M_r and Σ_r . However, in this case, the randomness comes from two sources: the finite design-sample set to make $\hat{h}(X)$ random and the finite test-sample set to make $\hat{p}_i(X)$ random. Since these two affect the error differently, we need to discuss their effects separately.

Effect of test samples. When a finite number of samples is available for testing a given classifier, an *error-counting* procedure is the only feasible possibility in practice. That is, each sample is tested by the classifier and the number of misclassified samples is counted. Then,

$$E_t\{\hat{\varepsilon}_r\} = \varepsilon_r \quad \text{and} \quad \text{Var}_t\{\hat{\varepsilon}_r\} = \frac{\varepsilon_r(1 - \varepsilon_r)}{\mathcal{N}_r} \quad (r = 1, 2) \quad (4.11)$$

where E_t and Var_t indicate that the expectations are taken with respect to test samples, and $\mathcal{N}_r$ is the number of test samples from ω_r . This is an unbiased estimate. Furthermore, (4.11) is valid regardless of functional forms for $h(X)$ and $p_r(X)$.

Effect of design samples. When a finite number of design samples is used to compute $\hat{M}_r$, $\hat{\Sigma}_r$ and then $\hat{h}(X)$, $E_d\{\hat{\varepsilon}_r\}$ and $\text{Var}_d\{\hat{\varepsilon}_r\}$ can be obtained through (2.23) and (2.24) for given test distributions, $p_r(X)$, where E_d and Var_d indicate the expectations with respect to design samples. The resulting bias is

$$E_d\{\hat{\varepsilon}\} \cong \varepsilon + \frac{\nu}{N} \quad (4.12)$$

where $\varepsilon = P_1\varepsilon_1 + P_2\varepsilon_2$, and ν is determined by the underlying distributions of design samples, given test distributions and the functional form of $h(X)$. The number of design samples is denoted by N ($= N_1 = N_2$) and distinguished from the test sample size $\mathcal{N}$. Although ν is a complicated function and can be computed

explicitly only for simple special cases, ν can be obtained empirically by the estimation procedure of (4.8).

When $h(X)$ is the quadratic classifier of (2.22) with $\Sigma_1 = \Sigma_2 = I$ and $P_1 = P_2$, ν becomes

$$\nu_q \cong \frac{1}{4\sqrt{2\pi M^T M}} e^{-M^T M/8} \left[n^2 + \left(1 + \frac{M^T M}{2} \right) n + \left(\frac{(M^T M)^2}{16} - \frac{M^T M}{2-1} - 1 \right) \right] \quad (4.13)$$

where $M = M_2 - M_1$.

On the other hand, when $h(X)$ is the linear classifier of (2.17) with $\Sigma_1 = \Sigma_2 = I$ and $P_1 = P_2$, ν becomes

$$\nu_\ell = \frac{1}{4\sqrt{2\pi M^T M}} e^{-M^T M/8} \left[\left(1 + \frac{M^T M}{4} \right) n - 1 \right]. \quad (4.14)$$

When $\Sigma_1 = \Sigma_2$, the quadratic $h(X)$ of (2.22) becomes the same as the linear $h(X)$ of (2.17). However, when the estimated covariance matrices, $\hat{\Sigma}_1 \neq \hat{\Sigma}_2$, are used, $\hat{h}(X)$ of (2.22) differs from that of (2.17). As a result, $E\{\Delta\varepsilon\}$ for the quadratic classifier is proportional to n^2/N while $E\{\Delta\varepsilon\}$ for the linear classifier tends to n/N when n gets large. This implies that many more samples are needed to properly design a quadratic classifier than a linear classifier.

More generally, (4.6) suggests that the bias could be proportional to n^2 because of the double summation of the last term. This is due to the fact that n^2 correlations are estimated. This number could be significantly reduced, if we assume a structure of the covariance matrix and estimate a smaller number of parameters.

As for $\text{Var}_d\{\hat{\varepsilon}\}$, we have

$$\text{Var}_d\{\hat{\varepsilon}\} \propto \begin{cases} \frac{1}{N^2} & \text{for the Bayes classifier} \\ \frac{1}{N} & \text{otherwise} \end{cases} \quad (4.15)$$

Effect of independent design and test samples. When both design and test samples are finite and they are independent, the bias and variance of $\hat{\varepsilon}$ are

$$E\{\hat{\varepsilon}\} \cong E_d\{\hat{\varepsilon}\} \quad (4.16)$$

$$\text{Var}\{\hat{\varepsilon}\} \cong \sum_{r=1}^2 P_r^2 \frac{E_d\{\hat{\varepsilon}_r\}[1 - E_d\{\hat{\varepsilon}_r\}]}{\mathcal{N}_r} + \text{Var}_d\{\hat{\varepsilon}\} \quad (4.17)$$

where E and Var indicate that the expectations are taken with respect to both design and test samples. Note that $\text{Var}_t\{\hat{\varepsilon}_r\}$ of (4.11) is obtained from the first term of (4.17) by replacing $E_d\{\hat{\varepsilon}_r\}$ by ε_r . Since $E_d\{\hat{\varepsilon}_r\} \cong \varepsilon_r + \nu_r/N_r$ we can conclude as follows:

- (1) the bias of the classification error comes entirely from the finite design set, and
- (2) the variance comes predominantly from the finite test set.

4.3. *Holdout, Leave-One-Out, and Resubstitution Methods*

When only one set of samples is available and the performance of the specified classifier is to be estimated, we need to decide how to divide the samples into two groups, design and test.

Upper and lower bounds of the Bayes error. In general, the classification error is a function of two sets of data, the design set $\mathcal{P}_D$ and the test set $\mathcal{P}_T$, and may be expressed as $\varepsilon(\mathcal{P}_D, \mathcal{P}_T)$ where ε is an operator to compute the Bayes error. We assume that both $\mathcal{P}_D$ and $\mathcal{P}_T$ are drawn from the same set of underlying distributions $\mathcal{P} = \{p_1(X), p_2(X)\}$. If $\mathcal{P}$ is used for design, the resulting classifier is the Bayes which produces the Bayes error by testing $\mathcal{P}$. That is, the Bayes error is expressed by $\varepsilon(\mathcal{P}, \mathcal{P})$. Letting $\hat{\mathcal{P}}_1$ and $\hat{\mathcal{P}}_2$ be two different sets of samples independently drawn from $\mathcal{P}$, $\varepsilon(\mathcal{P}, \mathcal{P})$ can be bounded as

$$E_{\hat{\mathcal{P}}_1} \{ \varepsilon(\hat{\mathcal{P}}_1, \hat{\mathcal{P}}_1) \} \leq \varepsilon(\mathcal{P}, \mathcal{P}) \leq E_{\hat{\mathcal{P}}_2} \{ \varepsilon(\hat{\mathcal{P}}_1, \hat{\mathcal{P}}_2) \}. \tag{4.18}$$

The rightmost term indicates that, as long as design and test sample sets are independent, the resulting error is larger than the Bayes in expectation with respect to test samples. The leftmost term suggests that, if the same set is used for both design and test, the resulting error is smaller than the Bayes in expectation. These procedures are called the *holdout (H)* and *resubstitute (R) methods* respectively.

The *H* method works well, if many data sets can be generated by a computer. However, in practice, with only one set of data, we need to divide the available data into two independent groups. This reduces the number of samples available for each of design and test. Also, it must be assured that the distributions of design and test samples are close. Another problem is how to allocate samples to design and test. This is normally done by balancing the bias due to the design sample size and the variance due to the test sample size.

The *leave-one-out (L) method* alleviates the above difficulties of the *H* method. In this method, one sample is excluded, the classifier is designed on the remaining $N - 1$ samples, and the excluded sample is tested by the classifier. This is repeated N times to test all N samples. The number of misclassified samples is counted to obtain the estimate of the error. Since each test sample is excluded from the design sample set, the design and test sets are independent. Also, since all N samples are tested and $N - 1$ samples are used for design, the available samples are more effectively utilized. Furthermore, we do not need to worry about dissimilarity between the design and test distributions. Although the *L* method requires N classifiers (one for each sample), these classifiers may be computed with little extra computer time as the perturbations from the classifier designed from N samples. This will be shown for the quadratic classifier next and for nonparametric cases later.

R and L methods for the quadratic classifiers. In the *R* method, all available samples, $X_i^{(r)}$ ($r = 1, 2; i = 1, \dots, N_r$), are used to compute $\hat{\mathbf{M}}_r$ and $\hat{\Sigma}_r$ of (4.5).

Then, the same samples are tested as to whether or not the following inequality is satisfied:

$$h_R(X_k) = \frac{1}{2} (X_k - \hat{M}_1)^T \hat{\Sigma}_1^{-1} (X_k - \hat{M}_1) - \frac{1}{2} (X_k - \hat{M}_2)^T \hat{\Sigma}_2^{-1} (X_k - \hat{M}_2) + \frac{1}{2} \ln(|\hat{\Sigma}_1|/|\hat{\Sigma}_2|) \stackrel{\omega_1}{\leqslant} t$$

for testing $X_k \in \{X_1^{(1)}, \dots, X_{N_1}^{(1)}, X_1^{(2)}, \dots, X_{N_2}^{(2)}\}$. (4.19)

Then, the number of misclassified samples is counted. This error is supposed to be smaller than the true error of the classifier.

On the other hand, in the L method, $X_k^{(1)} \in \omega_1$ is excluded from the computations of $\hat{M}_1$ and $\hat{\Sigma}_1$, and the modified $\hat{M}_1$ and $\hat{\Sigma}_1$ are used in (4.19) to test $X_k^{(1)}$. Similarly, $\hat{M}_2$ and $\hat{\Sigma}_2$ are modified for testing $X_k^{(2)} \in \omega_2$. The resulting quadratic equation $h_L(X_k)$ becomes

$$h_L(X_k) = h_R(X_k) + \begin{cases} +g(N_1, d_1^2(X_k)) & \text{for } X_k \in \omega_1 \\ -g(N_2, d_2^2(X_k)) & \text{for } X_k \in \omega_2 \end{cases} \quad (4.20)$$

where

$$g(N_r, d_r^2(X_k)) = \frac{1}{2} \frac{(N_r^2 - 3N_r + 1)d_r^2(X_k)/(N_r - 1) + N_r d_r^4(X_k)}{(N_r - 1)^2 - N_r d_r^2(X_k)} + \frac{1}{2} \ln \left[1 - \frac{N_r}{(N_r - 1)^2} d_r^2(X_k) \right] + \frac{n}{2} \ln \frac{N_r - 1}{N_r - 2} > 0 \quad (4.21)$$

and

$$d_r^2(X_k) = (X_k - \hat{M}_r)^T \hat{\Sigma}_r^{-1} (X_k - \hat{M}_r). \quad (4.22)$$

Equation (4.20) indicates that, for $X_k \in \omega_1$, $h_L(X_k)$ is larger than $h_R(X_k)$, and the chance of X_k being misclassified is increased. The same is true for $X_k \in \omega_2$. Therefore, the L error is always larger than the R error.

When the R method is used to count the error, $h_R(X_k)$ and $d_r^2(X_k)$ ($r = 1, 2$) must be computed for all $(N_1 + N_2)X_k$'s. The L method requires an additional computation of (4.21) for each k . However, since (4.21) is a scalar function, the computation time for this part is negligibly small. Thus, when $h_R(X_k)$ is computed and tested for each X_k , $h_L(X_k)$ is also computed and tested at the same time with little additional computer time.

5. Nonparametric Procedures

As Fig. 1 shows, nonparametric procedures are necessary for *error estimation* in both measurement and feature spaces and *data structure analysis* before a

parametric structure of the data is determined. Nonparametric procedures are based on the estimation of a density function without assuming any mathematical form.

5.1. Estimation of a Density Function

There are two approaches for density estimation: one is the *Parzen* approach and the other is the *k-nearest neighbor (NN)* approach. They have similar statistical properties with minor differences.

Parzen density estimate. When N samples $\mathbf{X}_1, \dots, \mathbf{X}_N$, are given, but no mathematical form can be assumed for the density function, the value of the density function at X may be estimated by

$$\hat{p}(X) = \frac{1}{N} \sum_{i=1}^N \kappa(X - \mathbf{X}_i) \tag{5.1}$$

where $\kappa(\cdot)$ is called a *kernel function*. In practice, selection of the kernel function is limited to either Gaussian or uniform particularly in a high-dimensional space. A more general form for $\kappa(\cdot)$ is

$$\kappa(X) = \frac{m\Gamma\left(\frac{n}{2}\right)\Gamma^{n/2}\left(\frac{n+2}{2m}\right)}{(n\pi)^{n/2}\Gamma^{n/2+1}\left(\frac{n}{2m}\right)} \cdot \frac{1}{r^n|A|^{1/2}} \exp \left[- \left\{ \frac{\Gamma\left(\frac{n+2}{2m}\right)}{n\Gamma\left(\frac{n}{2m}\right)} X^T (r^2 A)^{-1} X \right\}^m \right] \tag{5.2}$$

where $\Gamma(\cdot)$ is the gamma function. When $m = 1$ and ∞ , $\kappa(\cdot)$ of (5.2) are reduced to the Gaussian and uniform kernels respectively. The matrix A , which is called a *metric*, determines the shape of the hyperellipsoid, and r controls its size (both in the uniform case). Otherwise coefficients are selected to satisfy two conditions: $\int \kappa(X) dX = 1$ (which is required to satisfy $\int \hat{p}(X) dX = 1$) and $\int X X^T \kappa(X) dX = r^2 A$ (the covariance matrix of $\kappa(X)$).

The bias and variance of (5.1) are

$$E\{\hat{p}(X)\} \cong p(X) \left[1 + \frac{1}{2} \text{tr} \left\{ \frac{\nabla^2 p(X)}{p(X)} A \right\} r^2 \right] \tag{5.3}$$

$$\text{Var}\{\hat{p}(X)\} \cong \frac{wp(X)}{N} \text{ with } w = \int \kappa^2(X) dX. \tag{5.4}$$

The control parameters of the Parzen density estimate are m, r, A and N and their optimal choices could be found by minimizing $E\{[\hat{p}(X) - p(X)]^2\}$ which is $(\text{Bias})^2 + \text{Var}$. However, the optimal selection of parameters for density estimation does not coincide with the one for classification.

kNN density estimate. In this approach, the k th NN sample of X is found, and the distance d_k (or the corresponding volume v_k) is measured, where $d_k^2 = (X_{kNN} - X)^T A^{-1} (X_{kNN} - X)$, $v_k = c|A|^{1/2} d_k^n$, A is a metric, and c is a constant. Then, the

density estimate at X is

$$\hat{p}(X) = \frac{k-1}{N\mathbf{v}_k(X)}. \quad (5.5)$$

where $\mathbf{v}_k$ is a random variable and the function of X . Defining u as the probability of a sample falling within v_k , the density function of $\mathbf{u}$ is known as

$$p_u(u) = \frac{N!}{(k-1)!(N-k)!} u^{k-1}(1-u)^{N-k} \quad 0 \leq u \leq 1. \quad (5.6)$$

Since $u \cong v_k p(X)$ for a small v_k , the density function of $\mathbf{v}_k$ can be computed from (5.6). Thus, the bias and variance of the kNN density estimate are also obtained, resulting in

$$E\{\hat{p}(X)\} \cong p(X) \left[1 + \frac{1}{2} \text{tr} \left\{ \frac{\nabla^2 p(X)}{p(X)} A \right\} \left(\frac{k-1}{N c|A|^{1/2} p(X)} \right)^{2/n} \right] \quad (5.7)$$

$$\text{Var}\{\hat{p}(X)\} \cong \frac{p^2(X)}{k-2}. \quad (5.8)$$

The term $(\cdot)^{2/n}$ of (5.7) is d_k^2 by (5.5). Also, $\text{Var}\{\hat{p}(X)\}$ of (5.4) becomes $p^2(X)/k$ for a uniform kernel in which w is $1/v$. That is, the biases and variances of the Parzen and kNN density estimates are very similar. The kNN density estimate could be considered as the Parzen one with a uniform kernel whose kernel size is adjusted by $p(X)$. The control parameters of the kNN density estimate are k , A and N , and their optimal choices are found by minimizing $E\{\hat{p}(X) - p(X)\}^2$.

Moments of the kNN distance. Since the density function of $\mathbf{v}_k$ is known, the moments of $\mathbf{d}_k$ can be computed, resulting in

$$E\{\mathbf{d}_k^m\} \cong \frac{\Gamma^{m/n}(1+n/2)}{\pi^{m/2}|A|^{m/2n}} \cdot \frac{\Gamma(k+m/n)}{\Gamma(k)} \cdot \frac{\Gamma(N+1)}{\Gamma(N+1+m/n)} \int p^{1-m/n}(X) dX \quad (5.9)$$

where the integrals for some distributions with the covariance matrix Σ are

$$\text{Gauss: } (2\pi)^{m/2} |\Sigma|^{m/2n} (1-m/n)^{-n/2} \quad (5.10)$$

$$\text{Uniform: } (2\pi)^{m/2} |\Sigma|^{m/2n} \Gamma^{-m/n} (1+n/2)(1+n/2)^{m/2}. \quad (5.11)$$

When m/n is small in a high-dimensional space, and A is selected as $A = \Sigma$, $E\{\mathbf{d}_k^m\}$ is determined predominantly by n and m . The effects of k and N are minimal as $\Gamma(k+m/n) \cong \Gamma(k)$ and $\Gamma(N+1+m/n) \cong \Gamma(N+1)$. Also, $E\{\mathbf{d}_k^m|X\}$ is computed by (5.9) without taking the integration, but is little affected by $p(X)$ because of a small m/n in power. The variance of $\mathbf{d}_k$ is very small and all $\mathbf{d}_k$'s are close to the expected value.

Estimation of the local dimensionality. The ratio of two averaged kNN distances depends only on k and n , but not on N and $p(X)$ as follows:

$$\frac{E\{\mathbf{d}_{k+1}\}}{E\{\mathbf{d}_k\}} = \frac{E\{\mathbf{d}_{k+1}|X\}}{E\{\mathbf{d}_k|X\}} = 1 + \frac{1}{kn}. \quad (5.12)$$

The n computed from $\mathbf{d}_k$'s and $\mathbf{d}_{k+1}$'s by (5.12) depends only on neighboring information, and thus indicates the *local dimensionality* (or *intrinsic dimensionality*). Generally, the dimensionality plays a dominant role in determining the statistical properties of any nonparametric estimation. For example, $E\{\mathbf{d}_k^m\}$ of (5.9) with $A = \Sigma$ is predominantly determined by n and m . However, it must be kept in mind that the n of (5.9) means the local dimensionality but not the global one.

Very large number of classes. When the number of classes is very large as in the hundreds, we may consider class expected vectors M_i ($i = 1, \dots, L$) as random vectors drawn from a distribution, $p(M)$. The classification error between a pair of classes, ω_i and ω_j , is determined by the distance between M_i and M_j and the amounts of noises around these M 's. The overall error depends on how many neighboring classes contribute the error. On the other hand, the previous discussion indicates that, in a high-dimensional space, the kNN distance is not affected by k , L and $p(M)$. That is, each class is surrounded by many other neighboring classes with almost equal distances. Thus, almost-equal pairwise errors are added up to form the total error which could become large. Classification of a very large number of classes must be handled with special care. It is not enough to confirm that each pair of classes can be classified with a reasonably small error.

5.2. Classification

Parzen classifier. Substituting the Parzen density estimates into the likelihood ratio classifier of (2.6),

$$\hat{h}(X) = -\ln \frac{\hat{p}_1(X)}{\hat{p}_2(X)} = -\ln \frac{\frac{1}{N_1} \sum_{i=1}^{N_1} \kappa_1(X - X_i^{(1)})}{\frac{1}{N_2} \sum_{i=1}^{N_2} \kappa_2(X - X_i^{(2)})} \underset{\omega_2}{\overset{\omega_1}{\gtrless}} t. \quad (5.13)$$

This is called the *Parzen classifier* and can be used to classify X , when a set of samples $\{X_1^{(1)}, \dots, X_{N_1}^{(1)}, X_1^{(2)}, \dots, X_{N_2}^{(2)}\}$ is given. Each class may have a distinct kernel function.

In order to find the upper and lower bounds of the Bayes error, we may adopt the resubstitution (R) and leave-one-out (L) methods for the Parzen classifier. In the R method, the same samples, $X_i^{(r)}$ ($r = 1, 2$; $i = 1, \dots, N_r$), are tested by (5.13), and the number of misclassified samples is counted. On the other hand, when $X_\ell^{(1)}$ is tested in the L method, $X_\ell^{(1)}$ is excluded to form the Parzen density estimate of ω_1 . Therefore, the numerator of (5.13) must be replaced by

$$\hat{p}_{1L}(X_\ell^{(1)}) = \frac{1}{N_1 - 1} \left[\sum_{i=1}^{N_1} \kappa_1(X_\ell^{(1)} - X_i^{(1)}) - \kappa_1(X_\ell^{(1)} - X_\ell^{(1)}) \right]. \quad (5.14)$$

The denominator stays the same as $\hat{p}_2(X_\ell^{(1)})$. Again, $X_\ell^{(1)}$ ($\ell = 1, \dots, N_1$) are tested and the number of misclassified samples is counted. Note that the amount

subtracted in (5.14), $\kappa_1(0)$, does not depend on ℓ . When an ω_2 sample is tested, the denominator of (5.13) is modified in the same way. It can be proved that $\hat{p}_{1L}(X_\ell^{(1)}) \leq \hat{p}_1(X_\ell^{(1)})$ if $\kappa(X) \leq \kappa(0)$ which is satisfied for the kernel functions of (5.2). Therefore, the tested sample has more of a chance to be misclassified in the L method than in the R method. Also, note that the L density estimate of (5.14) can be obtained from the R density estimate by simple scalar operations — subtracting $\kappa_1(0)$ and dividing by $N_1 - 1$. Therefore, the computation time needed to obtain both the L and R density estimates is almost the same as that needed for the R density estimate alone.

kNN classifier. Using the kNN density estimates of (5.5), the likelihood ratio classifier becomes

$$\hat{h}(X) = -\ln \frac{\hat{p}_1(X)}{\hat{p}_2(X)} = -n \ln \frac{d_{k_2}^{(2)}(X)}{d_{k_1}^{(1)}(X)} - \ln \frac{(k_1 - 1)N_2|A_2|^{1/2}}{(k_2 - 1)N_1|A_1|^{1/2}} \underset{\omega_2}{\overset{\omega_1}{\leq}} t \quad (5.15)$$

where $v_{k_r}^{(r)} = c|A_r|^{1/2}d_{k_r}^{(r)n}$ and $d_{k_r}^{(r)2}(X) = (X_{k_r NN}^{(r)} - X)^T A^{-1}(X_{k_r NN}^{(r)} - X)$. That is, in order to test X , the $k_1 NN$ from ω_1 , $X_{k_1 NN}^{(1)}$, and the $k_2 NN$ from ω_2 , $X_{k_2 NN}^{(2)}$, are found, the distances from X to these neighbors, $d_{k_1}^{(1)}$ and $d_{k_2}^{(2)}$, are measured, and $d_{k_1}^{(1)}$ and $d_{k_2}^{(2)}$ are inserted into (5.15) to test whether the left hand side is smaller or larger than t . For simplicity, $k_1 = k_2 = k$ is used in this chapter. Normally, the class covariance matrix is used for A_r and therefore $A_1 \neq A_2$.

The R and L methods of the kNN classifier are used to bound the Bayes error. In the R method, all samples are included in the list of design samples from which the kNN of the test sample is found, and the same samples are tested. When $X_\ell^{(1)}$ is tested, $X_\ell^{(1)}$ itself is the closest sample in the list. Therefore, $d_{k-1}^{(1)}(X_\ell^{(1)})$ is inserted into the denominator of (5.15) while $d_k^{(2)}(X_\ell^{(1)})$ is inserted into the numerator. On the other hand, when $X_\ell^{(1)}$ is tested in the L method, $X_\ell^{(1)}$ must be excluded from the list of design samples. Therefore, $d_k^{(1)}(X_\ell^{(1)})$ and $d_k^{(2)}(X_\ell^{(1)})$ are compared. Since $d_{k-1}^{(1)}(X_\ell^{(1)}) \leq d_k^{(1)}(X_\ell^{(1)})$, $X_\ell^{(1)}$ has more of a chance to be misclassified in the L method than in the R method. Also, note that in order to find the kNN , the distances to all samples are computed and compared. When $d_k^{(1)}(X_\ell^{(1)})$ is obtained, $d_{k-1}^{(1)}(X_\ell^{(1)})$ is also available. This means that the computation time needed to get both the L and R results is practically the same as the time needed for the R method alone. Similarly, for testing a ω_2 sample, $X_\ell^{(2)}$, $d_{k-1}^{(2)}(X_\ell^{(2)})$ and $d_k^{(2)}(X_\ell^{(2)})$ are compared with $d_k^{(1)}(X_\ell^{(2)})$ in the R and L method respectively.

Voting kNN classifier. Instead of selecting kNN from each class separately and comparing the distances, the kNN 's of a test sample are selected from the mixture of classes, and the number of neighbors from each class, k_i , among the kNN is counted. The test sample is then classified to the class represented by a majority

of the kNN 's. That is,

$$k_r = \max\{k_1, \dots, k_L\} \rightarrow X \in \omega_r \left(\sum_{i=1}^L k_i = k \right). \quad (5.16)$$

In order to avoid confusion between these two kNN procedures, we will call (5.16) the *voting kNN* procedure and (5.15) the *volumetric kNN* procedure.

For the voting kNN procedure, it is common practice to use the same metric to measure the distances to samples from all classes, although each class could use its own metric. Since the k_i 's are integers and a ranking procedure is used, it is hard to find a component of (5.16) analogous with the threshold of (5.15).

It can be shown that, with $t = 0$ in (5.15), the volumetric kNN and voting $(2k-1)NN$ procedures give identical classification results for the two-class problem using the same metric for both classes. For example, let k and $(2k-1)$ be 3 and 5 respectively. In the voting $5NN$ procedure, a test sample is classified to ω_1 , if 3, 4, or 5 of the $5NN$'s belong to ω_1 . This is equivalent to saying that the third NN from ω_1 is closer to the test sample than the third NN from ω_2 .

In the voting kNN classification for the two-class problem, k must be odd. Otherwise, $k_1 = k_2$ could occur and we cannot decide which class the test sample is classified to. This problem may be alleviated by introducing the concept of *rejection*. That is, when $k_1 = k_2$ occurs, the test sample is rejected, and not counted as misclassified one. As a result, the classification error becomes smaller than even the Bayes error. This happens because some of the to-be-misclassified samples are rejected and not counted as the error.

The *asymptotic* ($N_r = \infty$) performance of the voting kNN is known. For the two-class problem, the *risk* of the voting kNN classification given X , $r_k(X)$, is

$$r_{2k-1}(X) = \sum_{i=1}^k \frac{1}{i} \binom{2i-2}{i-1} \xi^i(X) + \frac{1}{2} \binom{2k}{k} \xi^k(X) \quad (5.17)$$

$$r_{2k}(X) = \sum_{i=1}^k \frac{1}{i} \binom{2i-2}{i-1} \xi^i(X) \quad (5.18)$$

where $\xi(X) = q_1(X)q_2(X)$. On the other hand, the *Bayes risk* given X is

$$\begin{aligned} r^*(X) &= \min[q_1(X), q_2(X)] = \frac{1}{2} - \frac{1}{2} \sqrt{1 - 4\xi(X)} \\ &= \sum_{i=1}^{\infty} \frac{1}{i} \binom{2i-2}{i-1} \xi^i(X). \end{aligned} \quad (5.19)$$

Using (5.17)–(5.19), it is not difficult to prove that these conditional risks satisfy the following inequalities

$$\frac{1}{2} r^* \leq r_2 \leq r_4 \leq \dots \leq r^* \leq \dots \leq r_3 \leq r_1 \leq 2r^*. \quad (5.20)$$

Taking the expectation of these risks with respect to $\mathbf{X}$, the corresponding errors can be obtained. Therefore, these errors also satisfy the inequalities of (5.20). Thus,

$$\frac{1}{2}\varepsilon^* \leq \varepsilon_{2NN} \leq \varepsilon_{4NN} \leq \cdots \leq \varepsilon^* \leq \cdots \leq \varepsilon_{3NN} \leq \varepsilon_{NN} \leq 2\varepsilon^* \quad (5.21)$$

where

$$\varepsilon^* = E\{r^*(\mathbf{X})\} \quad \text{and} \quad \varepsilon_{kNN} = E\{r_k(\mathbf{X})\}.$$

Equation (5.21) indicates that asymptotically ($N_r = \infty$) the Bayes error is bounded by the voting kNN errors; the upper bounds for odd k 's and the lower bounds for even k 's.

5.3. Selection of Parameters

As is seen in Fig. 1, one of the major objectives of nonparametric classification is to estimate the Bayes error. This is done by finding the upper and lower bounds, using the L and R methods. However, normally nonparametric estimates are heavily biased unless the parameters are carefully chosen, and the results might not bound the Bayes error. Simply increasing the sample size is not the way to reduce the biases. The required number of samples could be astronomical particularly in a high-dimensional space.

Bias of the voting kNN classification. The asymptotic kNN error is obtained by assuming $q_r(X_{\ell NN}) = q_r(X)$ where $X_{\ell NN}$ is the ℓNN of X . However, as (5.9) suggests, $X_{\ell NN}$ and X are not close in a high dimensional space. Using a better approximation, $q_r(X_{\ell NN}) \cong q_r(X) + \nabla^T q_r(X)(X_{\ell NN} - X) + \frac{1}{2} \text{tr}\{\nabla^2 q_r(X)(X_{\ell NN} - X)(X_{\ell NN} - X)^T\}$, we can compute the simplest case, the bias of the NN error, for a finite sample size, resulting in

$$E\{\hat{\varepsilon}_{NN}\} \cong \varepsilon_{NN} + \beta_1 E_X\{|A|^{-1/n} \text{tr}\{AB_1(X)\}\} \quad (5.22)$$

where ε_{NN} is the asymptotic NN error, A is the metric, $B_1(X)$ is a matrix determined by the underlying distributions, and β_1 is a constant related to N and n as follows:

$$\beta_1 = \frac{\Gamma^{2/n}(1 + n/2)\Gamma(1 + 2/n)}{n\pi} \cdot \frac{\Gamma(N + 1)}{\Gamma(N + 1 + 2/n)} \cong c_1(n)N^{-2/n}. \quad (5.23)$$

For given distributions (given $B_1(X)$ and n), the bias is controlled by N and A . However, the reduction of β_1 in (5.23) by increasing N is very slow for a large n . For example, for $n = 64$, $N^{-2/n}$ is 0.81, 0.65 and 0.52 for $N = 10^3$, 10^6 and 10^9 respectively. The optimal A should be obtained by minimizing $E_X\{\cdot\}$ of (5.22) for the global metric and $|A|^{-1/n} \text{tr}\{AB_1(X)\}$ for the local metric. However, since $B_1(X)$ is too complex to compute in practice, we do not know how to select A .

The above result can be extended to the $2NN$ error as

$$E\{\hat{\varepsilon}_{2NN}\} \cong \varepsilon_{2NN} + \beta_2 E_X\{[|A|^{-1/n} \text{tr}\{AB_2(X)\}]^2\} \tag{5.24}$$

$$\beta_2 = \left[\frac{\Gamma^{2/n}(1+n/2)}{n\pi} \right]^2 \frac{(1+4/n)\Gamma(1+4/n)}{(1+2/n)} \cdot \frac{\Gamma(N+1)}{\Gamma(N+1+4/n)} \cong c_2(n)N^{-4/n} \tag{5.25}$$

where $B_2(X)$ is another matrix determined by the underlying distributions. In (5.25), β_2 is proportional to $N^{-4/n}$ while β_1 is to $N^{-2/n}$. That is, as N increases, the $2NN$ error converges to its asymptotic value more quickly than the NN error – as if n were half as large. Also, note that β_2 is significantly smaller than β_1 , because $\Gamma^{2/n}/n\pi$ (0.071 for $n = 64$) is squared. Since their asymptotic errors are related by $\varepsilon_{NN} = 2\varepsilon_{2NN}$ from (5.17) and (5.18), a better estimate of ε_{NN} could be obtained by estimating ε_{2NN} first and doubling it.

Parzen classifiers. In the L method, design and test samples are independent, and the bias of the classification error comes from the finite design samples. In this case, $h(X)$ in (2.23) and (2.24) is replaced by $\hat{h}(X)$ of (5.13) in which the bias and variance of $\hat{p}_r(X)$ are given in (5.3) and (5.4). Therefore, we can express the bias of the error of the Parzen classifier in terms of r and N as

$$E\{\Delta\hat{\varepsilon}\} \cong a_1r^2 + a_2r^4 + a_3 \frac{r^{-n}}{N} \tag{5.26}$$

where a_1 , a_2 and a_3 are determined by the underlying distributions, the metrics A_1 and A_2 for ω_1 and ω_2 , the parameter to control the shape of the kernel m , and the threshold t in (5.13). Recall in (5.3) and (5.4) that the bias of the Parzen density estimate is a function of $\nabla^2 p(X)$, A and r^2 while the variance is a function of $p(X)$, N and w (which is a function of r , A and m).

The a_1r^2 and a_2r^4 terms indicate how biases in the density estimates influence the performance of the classifier, while the a_3r^{-n}/N term reflects the role of the variance of the density estimates. For small values of r , the variance term dominates (5.26), and the observed error rates are significantly above the Bayes error. As r grows, however, the variance term decreases while the a_1r^2 and a_2r^4 terms play an increasingly significant role. Thus, for a typical plot of the observed error rate versus r , $\hat{\varepsilon}$ decreases for small values of r until a minimum point is reached, and then increases as the bias terms of the density estimates become more significant.

The r^{-n} in the third term of (5.26) for a small r and a large n is astronomically large (for example, $r^{-n} = 1.8 \times 10^{19}$ for $r = 0.5$ and $n = 64$). It is futile to attempt to bring down this term by selecting a large N and reducing a_3 . The optimal r can be obtained by taking the derivative of (5.26) with respect to r and equating it to zero. The more practical solution is to compute the L and R errors for the various values of r , plot the curves, and find the optimal r .

As r increases, the biases of both L and R errors increase, and they do not bound the Bayes error. In order to reduce this bias, we must make a_1 and a_2 of (5.26) small by selecting parameters, t , A_1 , A_2 and m .

- (1) *Selection of the decision threshold t .* Changing t is a very effective way to reduce a_1 and a_2 . Since a_1 and a_2 are very complicated functions, it is more practical to find the optimal value for t experimentally. The different optimal value for t must be computed for each selection of r . Although better but more complex procedures are available, a simple procedure to determine t is presented as follows:

For each value of r , find the value of t which minimizes the R error, and then use this value of t to find the L error. Since the selection of t is isolated from the actual values of the L estimate of the likelihood ratio, using this method helps to maintain the independence of the test operation from the design one.

- (2) *Selection of metrics, A_1 and A_2 .* Minimization of a_1 and a_2 with respect to A_1 and A_2 does not give any easy answer. Also, since A_1 and A_2 are $n \times n$ matrices, empirical try-and-error cannot be applied easily. An intuitive selection, which is normally used, is $A_r = \Sigma_r$. But, this is far from the optimal solution. Another alternative is $A_i = \Sigma_i - \gamma_i(X - M_i)(X - M_i)^T$ with a constant γ_i , the properties of which have not been studied extensively.

Even if $A_1 = \Sigma_r$ is a good selection of the metric, the estimation of Σ_r needs special care. That is, if the same sample set is used for both estimating Σ_r and computing the L and R errors, both L and R errors are severely and optimistically biased, and often they do not bound the Bayes error. In order to avoid this optimistic bias, we must use two independent sample sets, one for each of the above two operations. A better but more complex procedure is to exclude $X_\ell^{(r)}$ from estimating Σ_r , when $X_\ell^{(r)}$ is tested by the L and R classifiers.

- (3) *Selection of m .* As m increases, the L error curve is little affected. But, the R curve steadily rises closer to the Bayes error line until $m = 4$, and then stops to improve. It means that (1) both Gaussian and uniform kernels give the similar L errors, (2) the uniform kernel is better to compute the R error than the Gaussian kernel, and (3) the uniform-like kernels are obtained by selecting $m = 4$.

kNN classifiers. The parameters of the kNN classifier, k , t , A_1 and A_2 , are chosen in the same way as the ones of the Parzen classifier. That is, k and t are determined empirically. We do not know how to select A_1 and A_2 . Samples to compute the L and R errors must be independent from samples to estimate the metrics.

References

- [1] K. Fukunaga, *Introduction to Statistical Pattern Recognition*, Second Ed. (Academic Press, San Diego, CA, 1990).
- [2] R. O. Duda and P. E. Hart, *Pattern Classification and Scene Analysis* (Wiley, New York, 1973).

- [3] P. R. Devijver and J. Kittler, *Pattern Recognition: A Statistical Approach* (Prentice-Hall, Englewood Cliffs, New Jersey, 1982).
- [4] A. K. Agrawala (ed.), *Machine Recognition of Patterns* (IEEE Press, New York, 1977).
- [5] P. R. Krishnaiah and L. N. Kanal (eds.), *Handbook of Statistics 2: Classification, Pattern Recognition and Reduction of Dimensionality* (North-Holland Publ., Amsterdam, 1982).
- [6] T. Y. Young and K. S. Fu (eds.), *Handbook of Pattern Recognition and Image Processing* (Academic Press, San Diego, 1986).
- [7] B. V. Dasarathy (ed.), *Nearest Neighbor (NN) Norms: NN Pattern Classification Techniques* (IEEE Computer Society Press, Los Alamitos, CA, 1991).